Quest 2: The Axeperiment

Lord of the Wings · ranking Gimlock's alloys from hardest to softest

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION
Given head-to-head hardness results between alloys, find the order from hardest to softest, read it as a number, and add these numbers up.

INPUT
One line per group of alloys, such as `4<3 2<5 3>1 …`

OUTPUT
The checksum: the sum of the order-numbers.

WHAT CHANGES BETWEEN PARTS
How many comparison results are missing: none, exactly one, or many.

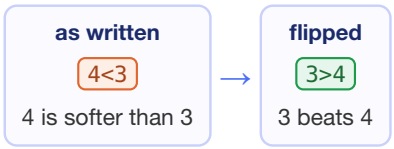
The story

The party stops at the Forge Mountain to recruit the dwarven blacksmith **Gimlock Ironstack**. He will only leave once his research on the hardness of new metal alloys is finished. His method is to hit two axe blades together and see which one gives way. You help him make sense of his notes.

Understanding the question

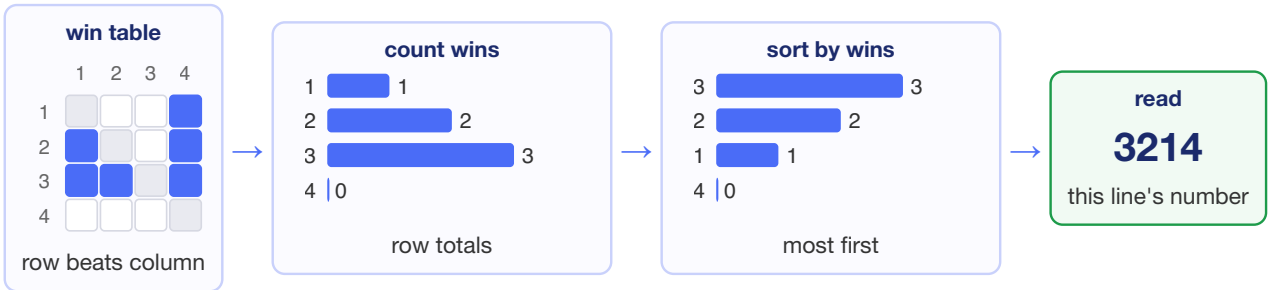
Reading one comparison

The pointy side of the symbol points at the **softer** alloy. Flip every `<` so the harder one is always on the left:



From comparisons to an order

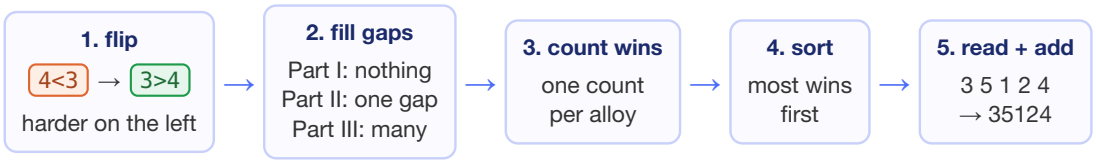
Each line is one group: order its alloys hardest → softest and read the order as one number. A 4-alloy example of my own, `2>1 3>1 3>2 1>4 2>4 3>4`:



> **Key insight:** with every pair known, the hardest alloy beats all n – 1 others, the next beats n – 2, …, the softest beats none. So **win count = position**. The checksum adds up the numbers from all lines.

Shared approach: from comparisons to a number

Every part runs the same pipeline. Only the **fill gaps** step differs:



PART I Every pair is compared

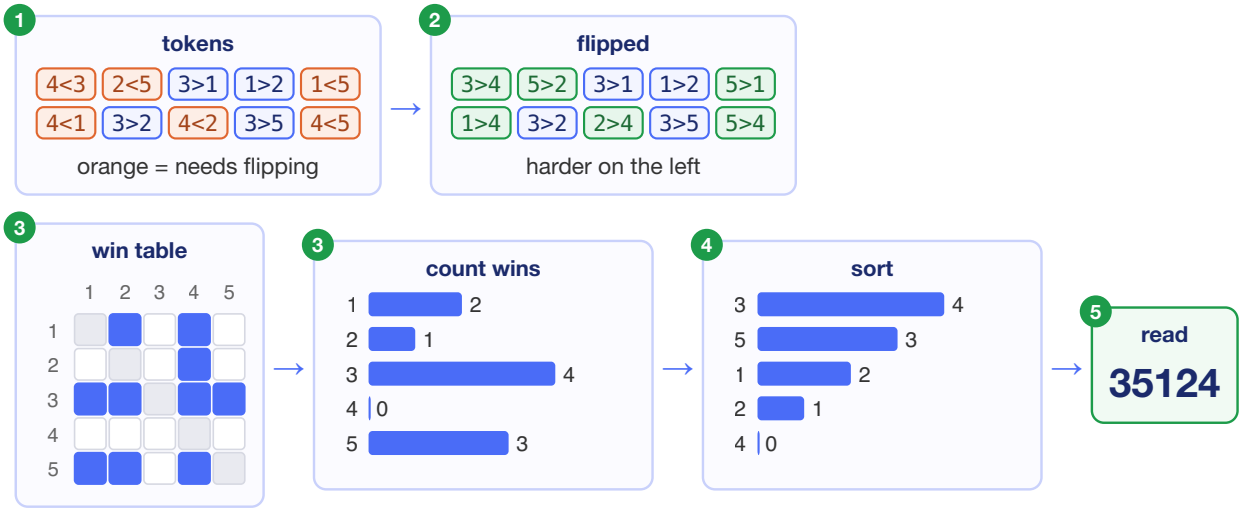
Each group has the result of every possible pair: with 5 alloys, 5 × 4 / 2 = 10 comparisons per line.

Step-by-step solution

1. Take one line (one group) and split it on spaces into comparison tokens.
2. Rewrite every token as `harder>softer`: `4<3` becomes `3>4`.
3. For each token, add 1 to the win count of the alloy on the left of `>`.
4. Sort the alloys by win count, most wins first. That is hardest to softest.
5. Glue the alloy numbers together into one number, such as 35124.

↻ *Do this for every line and add up the numbers. That is the checksum.*

Worked example: first sample line



Dots match the step numbers above.

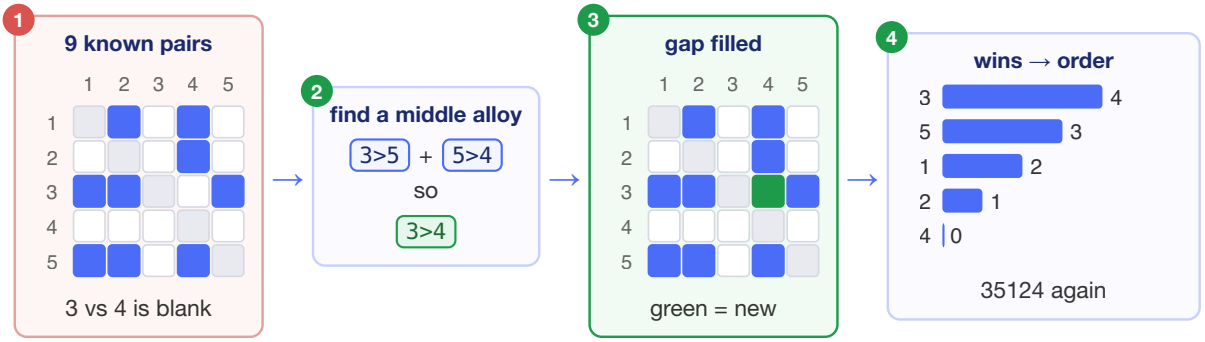
Line	Order	Number
4<3 2<5 3>1 1>2 1<5 4<1 3>2 4<2 3>5 4<5	3 5 1 2 4	35124
4>1 2>1 3>2 1<5 4>5 2<5 3>1 4>3 2<4 5<3	4 3 5 2 1	43521
5>1 5>2 4>1 2<4 3>2 3<4 1>2 3>5 1<3 4>5	4 3 5 1 2	43512
Checksum		**122157**

PART II Exactly one comparison is missing

Each group is missing one result. Hardness is transitive: if a > c and c > b, then a > b, so the gap can be filled through a third alloy.

Worked example: first sample line

Same line as Part I with `4<3` removed. Alloys 3 and 4 were never compared:



Dots match the step numbers below. The real input has 9 alloys per group, so its numbers are 9 digits long.

The sample is the Part I sample with one comparison removed from each line, so the checksum is still **122157**.

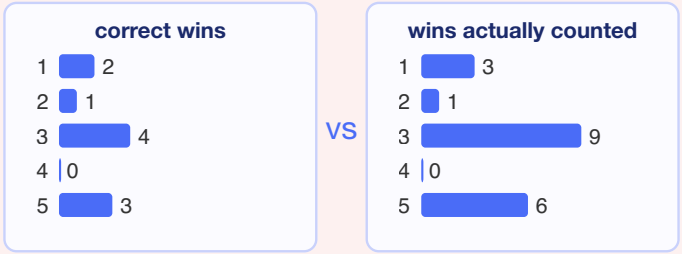
Step-by-step solution

1. Parse and rewrite the tokens as in Part I. One pair is missing.
2. For every pair of alloys (a, b), look for a third alloy c with `a>c` and `c>b`. If you find one, add `a>b`. Check the other direction (`b>c>a`) too.
3. The missing pair is now filled in.
4. Count wins, sort, glue the numbers together and add them up, as in Part I.

How my solution works

For every pair (a, b) it tries every other alloy c as the middle step, in both directions. Only one result is missing, so some middle alloy always links it. Then it counts wins as in Part I.

A subtle detail worth knowing: the solution adds `a>b` **every time** it finds a middle alloy, even if it was already known, so the win counts get inflated (19 comparisons instead of 10 on the first sample line):



The **order is the same**, because stronger alloys always pick up at least as many duplicates as weaker ones. It works by luck, not design. Part III keeps a set of pairs (each stored once), which avoids this.

PART III Many comparisons missing

Only some results are known, e.g. `5>1 3<4 1>2 3>5` (4 of 10 pairs). Knowing 4 > 2 needs the chain 4 > 3 > 5 > 1 > 2: you need every result that follows from chains of any length, the **transitive closure**.

Step-by-step solution

1. Parse and rewrite the tokens. Store them in a **set** of known pairs.
2. Take each alloy c in turn (1, 2, 3, …), **before** choosing a and b: for every a with `a>c` known, and every b with `c>b` known, add `a>b` to the set.
3. After the last c, every pair that follows from any chain is in the set (the transitive closure).
4. Count wins, sort, glue the numbers together and add them up, as in Part I.

How my solution works

Keep a set of known pairs. Take each alloy c in turn as the **middle**: everyone above c now beats everyone below c. After the last c, every chain has been followed. Third sample line, `5>1 3<4 1>2 3>5`:



Dots match the step numbers above. Green squares are pairs added for that middle alloy.

> **Why the middle alloy is chosen first:** at c = 5 the solution uses `5>2`, which only appeared at c = 1. Handling middles one after another lets each round build on the chains found before it. After middles 1 … k, every chain whose middle alloys are all ≤ k is covered.

The three sample lines give 35124, 43521 and 43512, so the checksum is again **122157**.

Summary

Part	Missing	How the gaps are filled	Approach
I	none	nothing to fill; just count wins	count wins, sort
II	exactly 1	one middle alloy is enough	try every middle alloy for every pair (duplicates are harmless)
III	many	full transitive closure	set of pairs; middle alloy chosen first (Floyd–Warshall)

> Another way to see Part III: draw an arrow a → b for "a is harder than b". The answer is the unique **topological order** of that graph, so Kahn's algorithm would also solve it.